

Coloration de graphes

Ludivine PATEUX–MORIN

17939

Avec Tudwal AUDREN et Philippe SUARD

- 1 Définitions & Résolution exacte
- 2 Algorithmes heuristiques : intérêts et limites
- 3 Comparaison des programmes

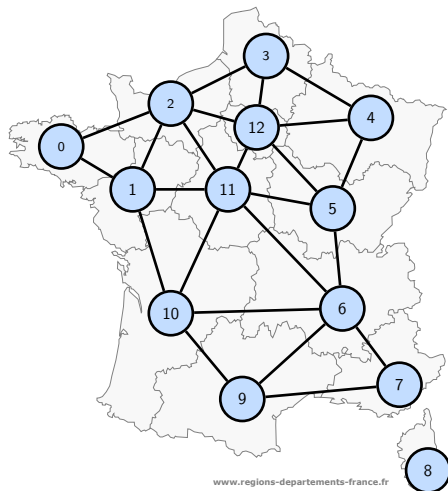
Coloration naïve de la carte

Combien de couleurs sont nécessaires pour colorer une carte ?



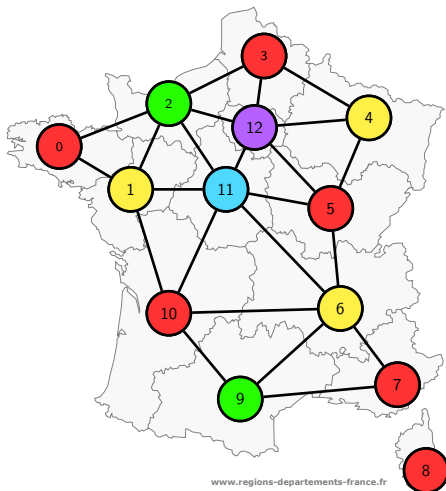
Coloration naïve de la carte

Combien de couleurs sont nécessaires pour colorer une carte ?



Coloration naïve de la carte

Combien de couleurs sont nécessaires pour colorer une carte ?



Plusieurs questions se posent :

- comment colorer un graphe avec un minimum de couleurs ?

Plusieurs questions se posent :

- comment colorer un graphe avec un minimum de couleurs ?
- comment résoudre ce problème efficacement avec un algorithme ?

Sommaire

1 Définitions & Résolution exacte

2 Algorithmes heuristiques : intérêts et limites

3 Comparaison des programmes

Partie I — Définitions

Définitions 1

- **coloration** = affectation d'une couleur à chaque sommet
- coloration **valide** = deux sommets **adjacents** sont toujours de **couleurs différentes**

Partie I — Définitions

Définitions 1

- **coloration** = affectation d'une couleur à chaque sommet
- coloration **valide** = deux sommets **adjacents** sont toujours de **couleurs différentes**

Définitions 2

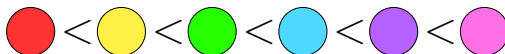
- C utilise k couleurs différentes $\Rightarrow G$ est **k -colorable**
- $\chi(G)$ = la **valeur minimale** telle que G soit $\chi(G)$ -colorable
 $\chi(G)$ est appelé **nombre chromatique**

Pour la suite

- Sommets numérotés de 0 à $n - 1$

Pour la suite

- Sommets numérotés de 0 à $n - 1$
- Couleurs représentées par des entiers
- On ordonne arbitrairement les couleurs pour la présentation :



Résolution exacte : limites

- Coloration exacte : **Branch-and-Bound**

Résolution exacte : limites

- Coloration exacte : **Branch-and-Bound**
 - Fonction de rejet modifiée au cours de la coloration

Résolution exacte : limites

- Coloration exacte : **Branch-and-Bound**
 - Fonction de rejet modifiée au cours de la coloration
- Coloration de graphes = pas de solution exacte en temps polynomial
 - À 25 sommets : temps d'exécution de B&B d'au plus 10 minutes
 - À 30 sommets : environ 18 minutes de temps de calcul !

Résolution exacte : limites

- Coloration exacte : **Branch-and-Bound**
 - Fonction de rejet modifiée au cours de la coloration
- Coloration de graphes = pas de solution exacte en temps polynomial
 - À 25 sommets : temps d'exécution de B&B d'au plus 10 minutes
 - À 30 sommets : environ 18 minutes de temps de calcul !
- Approche heuristique = alternative nécessaire en pratique

Sommaire

- 1 Définitions & Résolution exacte
- 2 Algorithmes heuristiques : intérêts et limites
- 3 Comparaison des programmes

Approche algorithmique intuitive

- Approche intuitive : First-Fit

Approche algorithmique intuitive

- Approche intuitive : First-Fit
 - On parcourt les sommets dans l'ordre lexicographique

Approche algorithmique intuitive

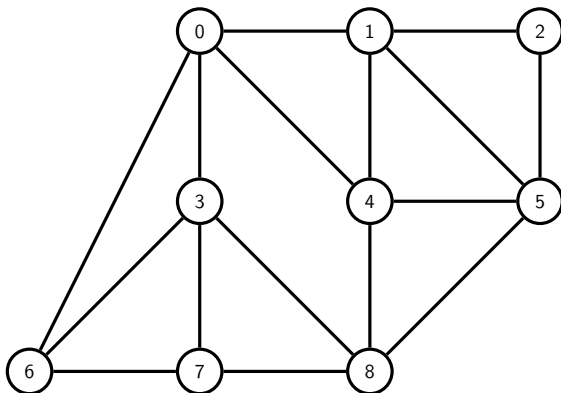
- Approche intuitive : First-Fit
 - On parcourt les sommets dans l'ordre lexicographique
 - On affecte la plus petite couleur disponible

Approche algorithmique intuitive

- Approche intuitive : First-Fit

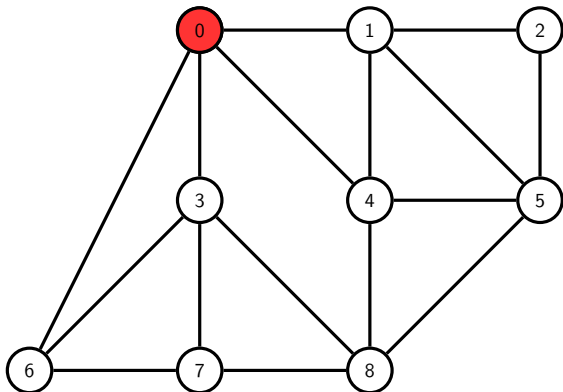
- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :



Approche algorithmique intuitive

- Approche intuitive : First-Fit
 - On parcourt les sommets dans l'ordre lexicographique
 - On affecte la plus petite couleur disponible
- Exemple :

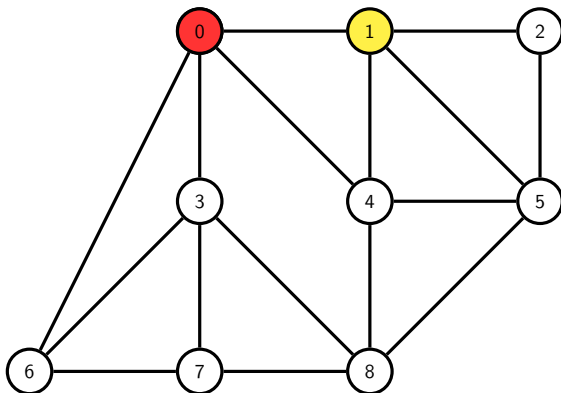


Approche algorithmique intuitive

- Approche intuitive : First-Fit

- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :

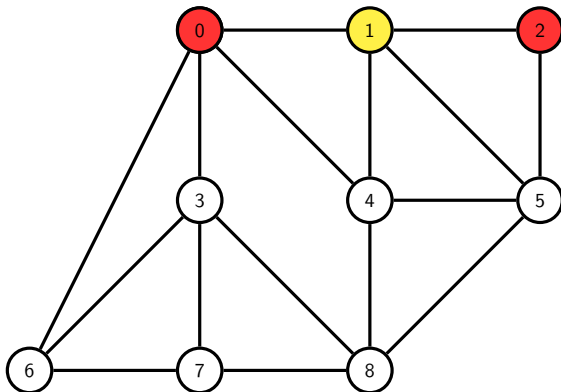


Approche algorithmique intuitive

- Approche intuitive : First-Fit

- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :

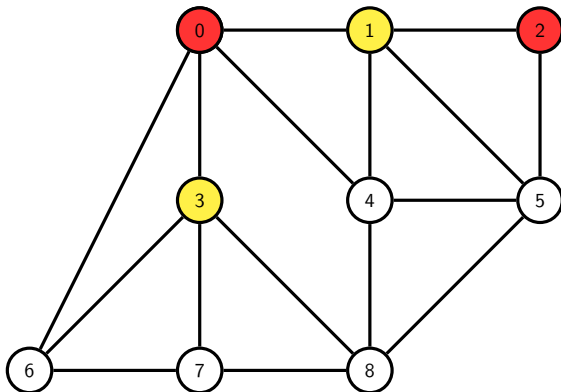


Approche algorithmique intuitive

- Approche intuitive : First-Fit

- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :

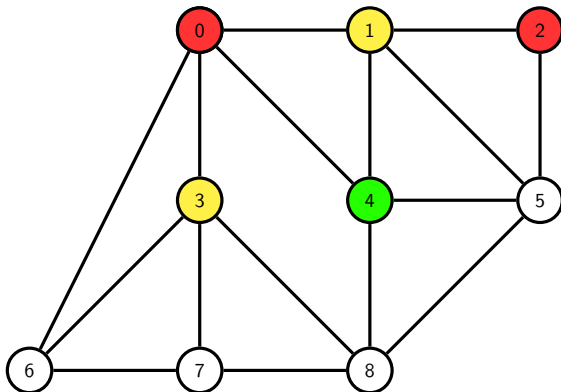


Approche algorithmique intuitive

- Approche intuitive : First-Fit

- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :

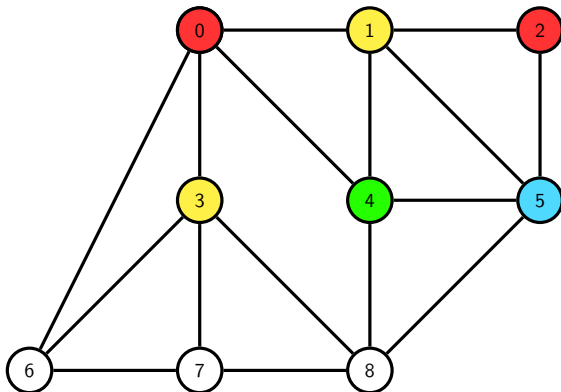


Approche algorithmique intuitive

- Approche intuitive : First-Fit

- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :

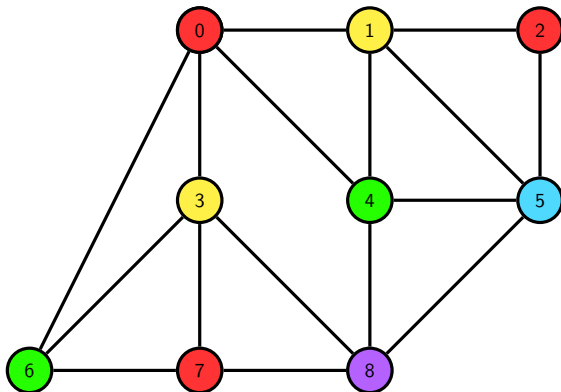


Approche algorithmique intuitive

- Approche intuitive : First-Fit

- On parcourt les sommets dans l'ordre lexicographique
- On affecte la plus petite couleur disponible

- Exemple :



First-fit

- Implémentation simple en $\mathcal{O}(n + m)$

First-fit

- Implémentation simple en $\mathcal{O}(n + m)$
- L'ordre de traitement peut changer drastiquement la coloration renvoyée

First-fit

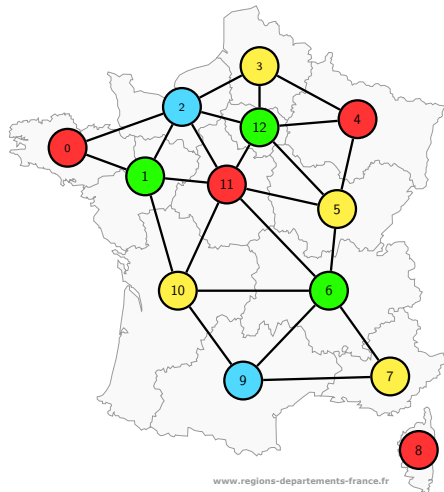
- Implémentation simple en $\mathcal{O}(n + m)$
- L'ordre de traitement peut changer drastiquement la coloration renvoyée
- **SI** on ordonne bien les sommets, First-Fit renvoie une coloration exacte

Théorème

Pour un graphe G , il existe un ordre de traitement des sommets tel que la coloration C donnée par l'algorithme First-Fit est exacte.

Intuition de la preuve

- On colore tous les sommets rouges
- Puis tous les jaunes
- etc...
- Chaque sommet sera recoloré avec une plus petite couleur que celle de la coloration exacte



Welsh-Powell

- Nouveau choix de traitement des sommets :

Welsh-Powell

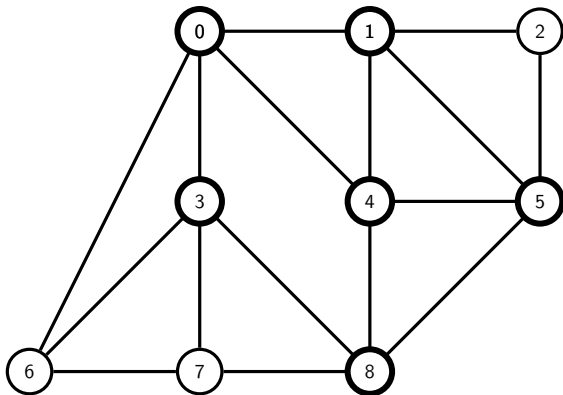
- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)

Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)

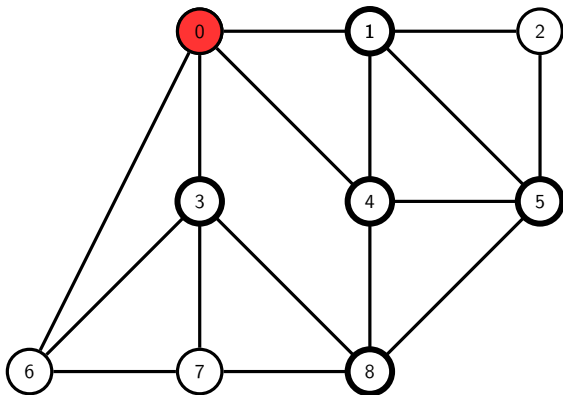
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



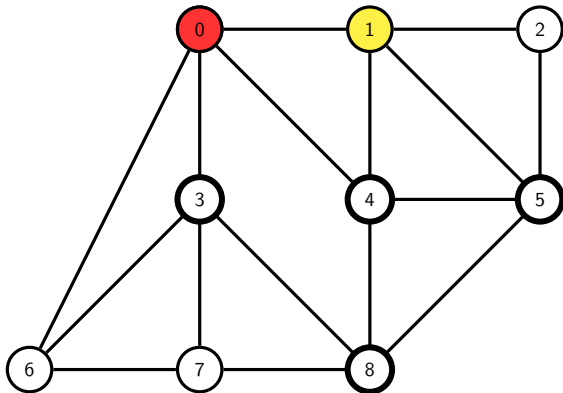
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



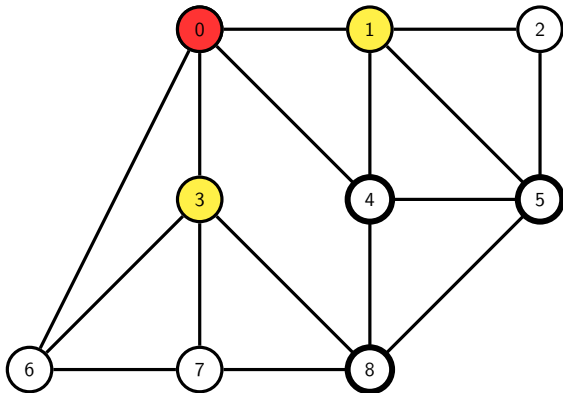
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



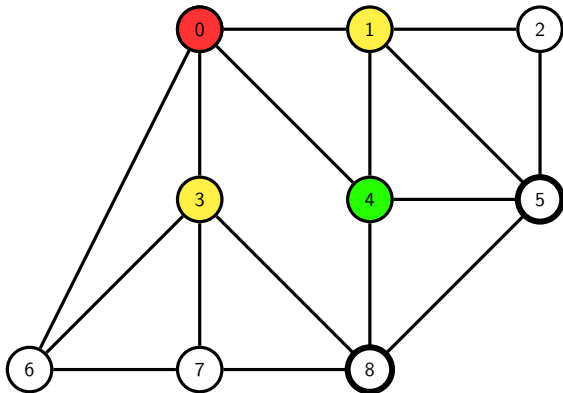
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



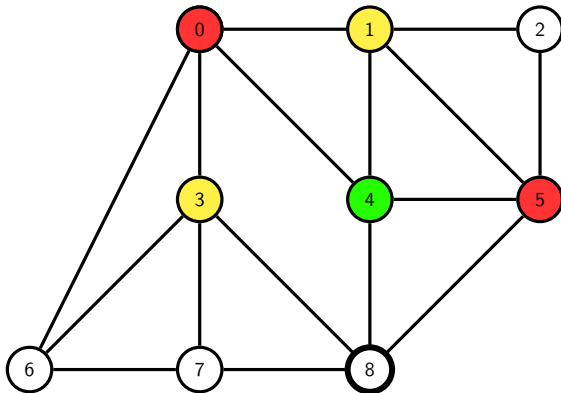
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



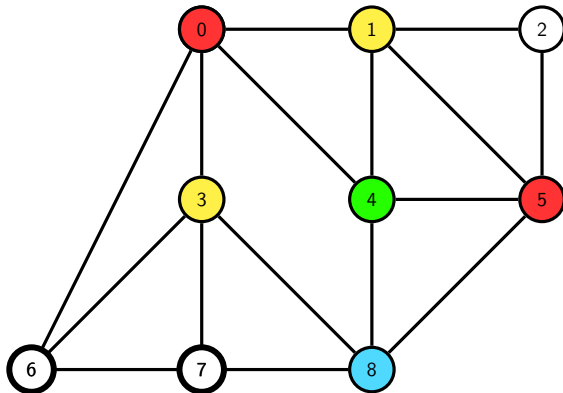
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



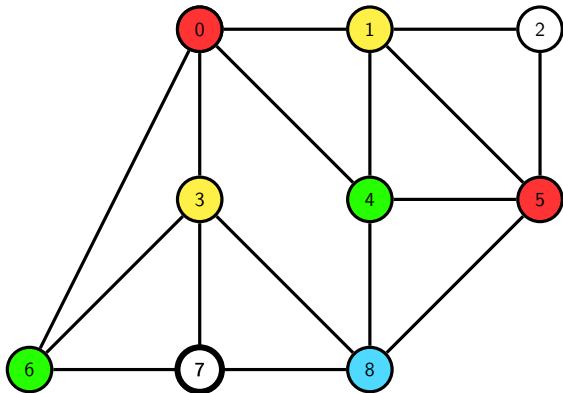
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



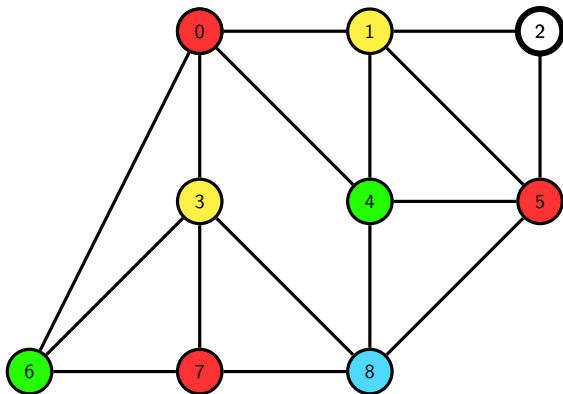
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



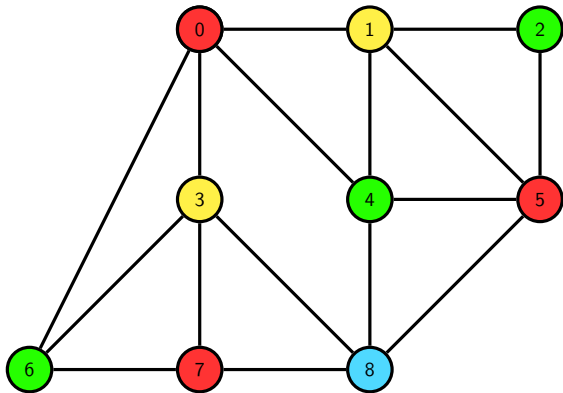
Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :



Welsh-Powell

- Nouveau choix de traitement des sommets :
 - **degré** (décroissant)
 - ordre lexicographique (croissant)
 - on affecte toujours la plus petite couleur disponible
- Exemple :

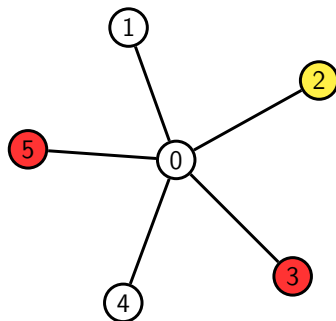


DSATUR

- Nouvel ordre de traitement :

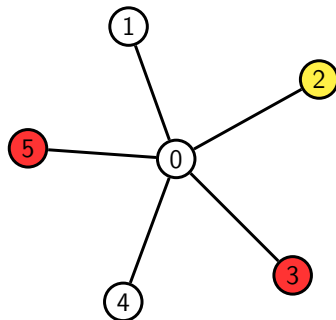
DSATUR

- Nouvel ordre de traitement :
 - **degré de saturation** (décroissant)



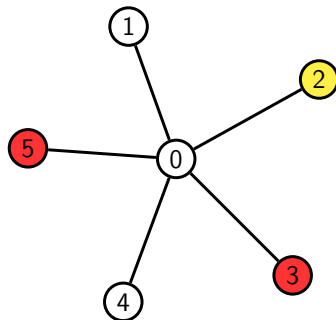
DSATUR

- Nouvel ordre de traitement :
 - **degré de saturation** (décroissant)
 - degré (décroissant)

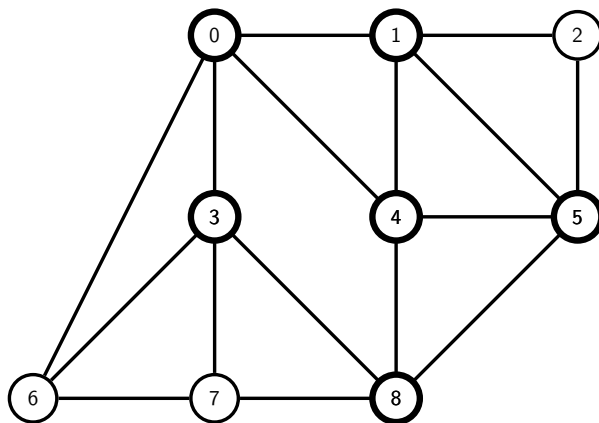


DSATUR

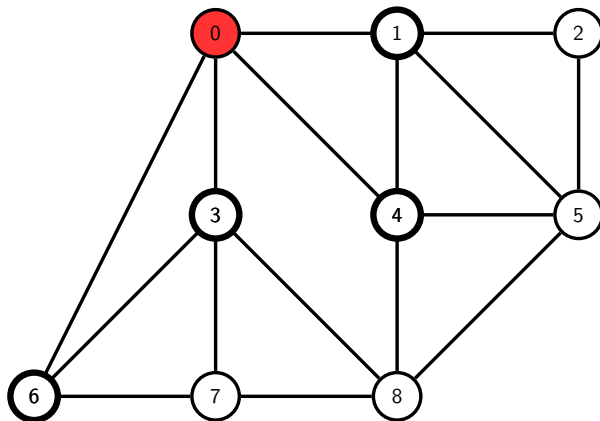
- Nouvel ordre de traitement :
 - **degré de saturation** (décroissant)
 - degré (décroissant)
 - ordre lexicographique (croissant)



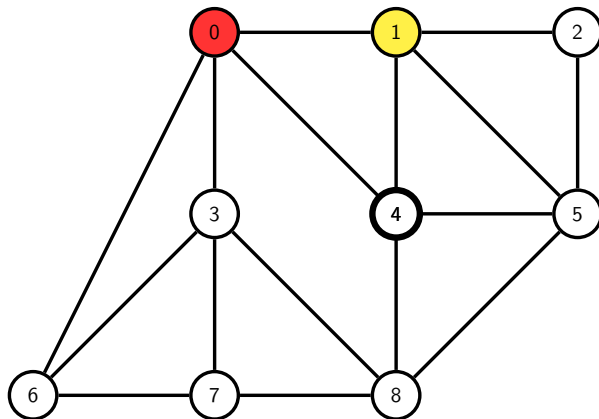
DSATUR : exemple



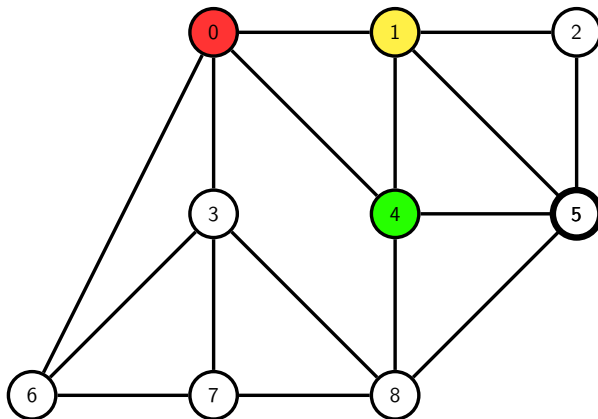
DSATUR : exemple



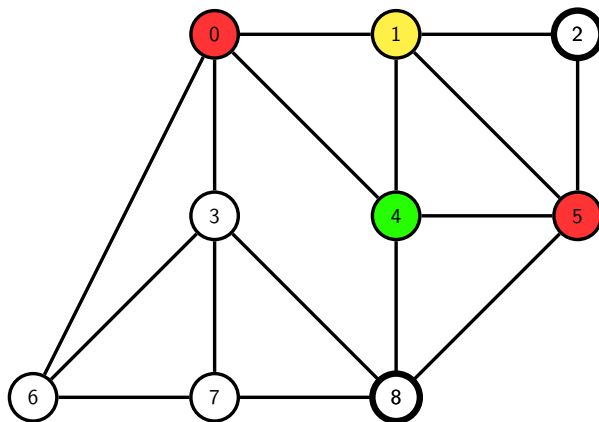
DSATUR : exemple



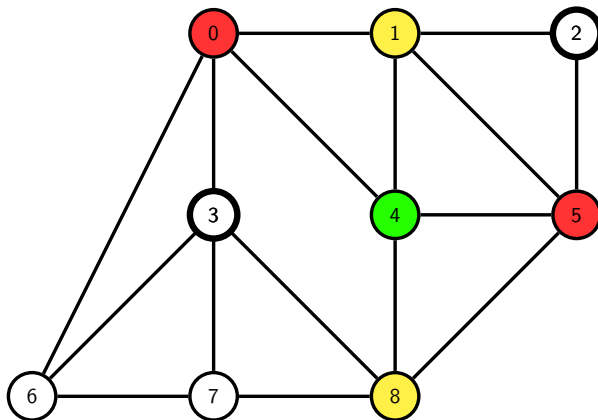
DSATUR : exemple



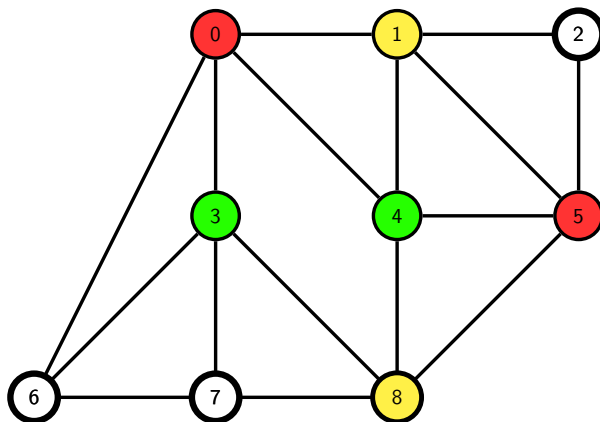
DSATUR : exemple



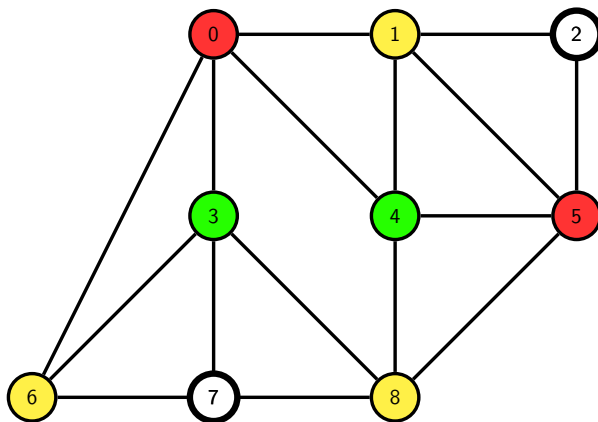
DSATUR : exemple



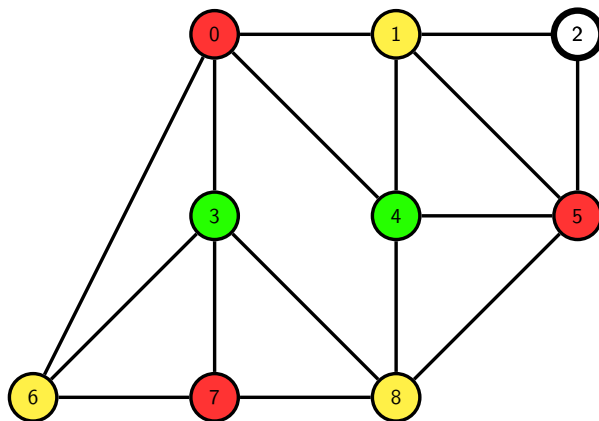
DSATUR : exemple



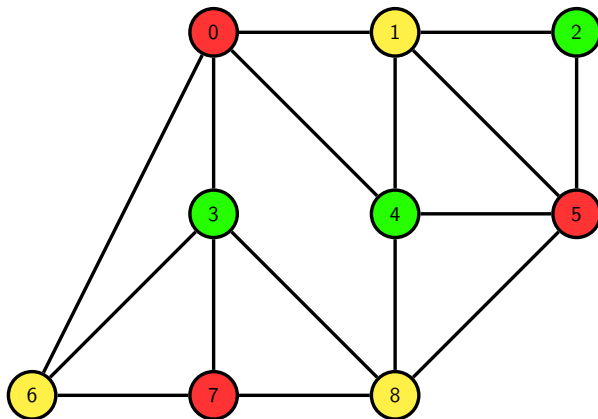
DSATUR : exemple



DSATUR : exemple

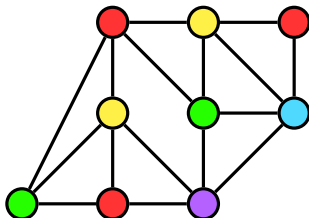


DSATUR : exemple



Comparaison des trois colorations

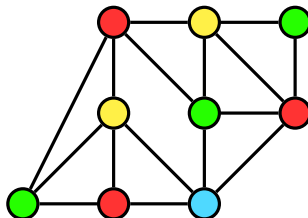
First-Fit



5-coloration

$$\mathcal{O}(n+m)$$

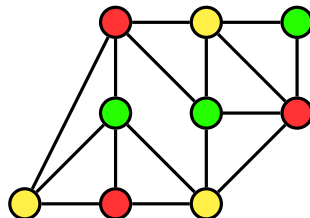
Welsh-Powell



4-coloration

$$\mathcal{O}(n \log(n) + m)$$

DSATUR



3-coloration

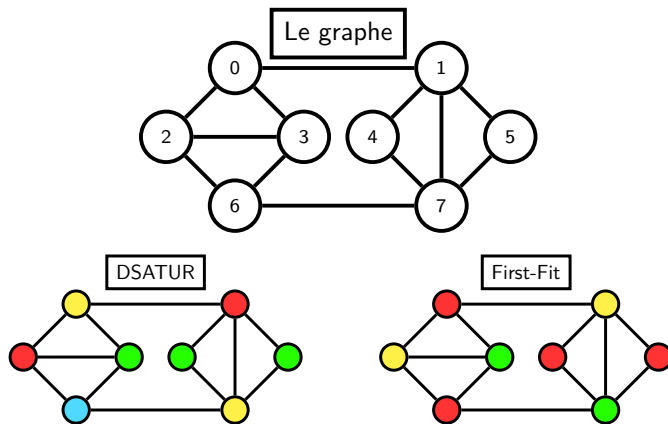
$$\mathcal{O}(n^2) \text{ ou } \mathcal{O}((n+m) \log(n))$$

DSATUR : premier problème

- DSATUR ne fournit pas toujours une coloration exacte :

DSATUR : premier problème

- DSATUR ne fournit pas toujours une coloration exacte :

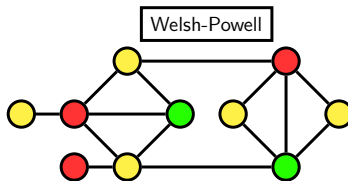
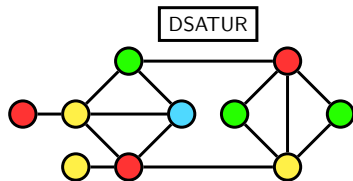
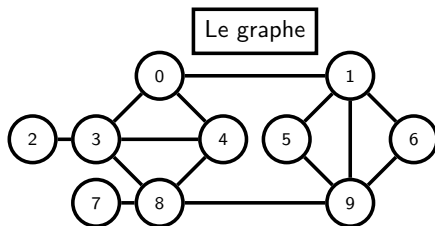


DSATUR : second problème

- DSATUR n'améliore même pas tout le temps Welsh-Powell

DSATUR : second problème

- DSATUR n'améliore même pas tout le temps Welsh-Powell



Sommaire

- 1 Définitions & Résolution exacte
- 2 Algorithmes heuristiques : intérêts et limites
- 3 Comparaison des programmes

Facteurs de comparaison

Caractéristiques de comparaison :

Facteurs de comparaison

Caractéristiques de comparaison :

- **temps** de calcul

Facteurs de comparaison

Caractéristiques de comparaison :

- **temps** de calcul
- **nombre de couleurs** utilisées

Facteurs de comparaison

Caractéristiques de comparaison :

- **temps** de calcul
- **nombre de couleurs** utilisées

Paramètres variables :

Facteurs de comparaison

Caractéristiques de comparaison :

- **temps** de calcul
- **nombre de couleurs** utilisées

Paramètres variables :

- **taille** des graphes

Facteurs de comparaison

Caractéristiques de comparaison :

- **temps** de calcul
- **nombre de couleurs** utilisées

Paramètres variables :

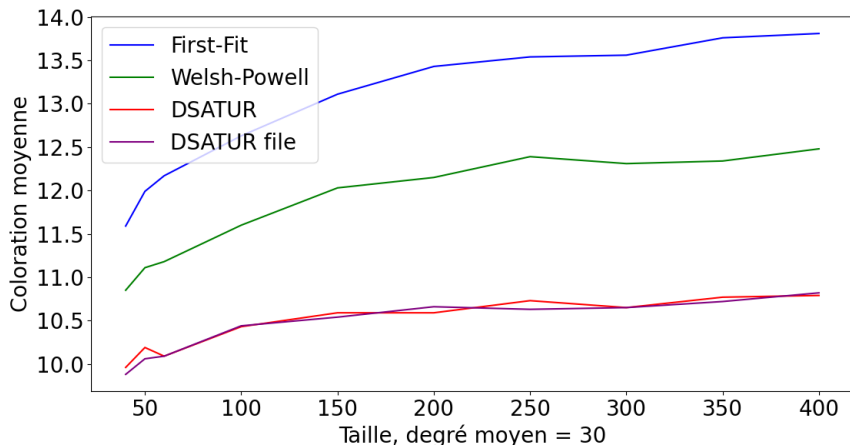
- **taille** des graphes
- **degré moyen** des graphes

Analyse des données

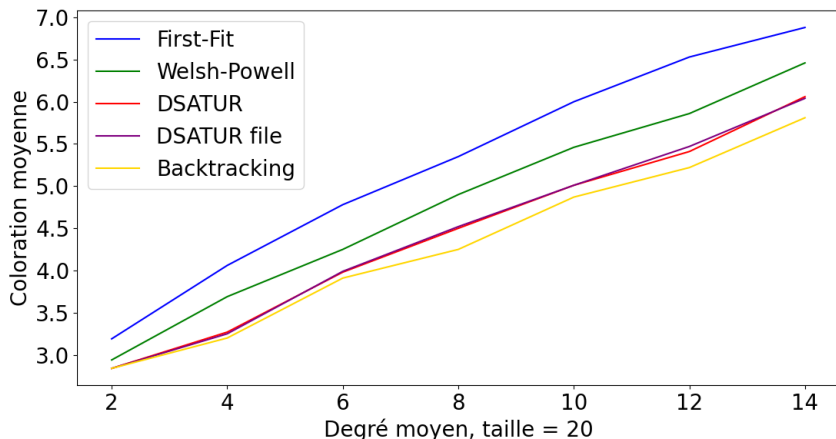
- Génération des graphes en Ocaml
- Collecte des données sur tableur
- Affichage des données en Python

	A	B	C	D	E	F	G	H	I	J	K	
1	Temps F-F	Col° F-F	Temps W-P	Col° W-P	Temps DSAT	Col° DSAT	Temps DSAT file	Col° DSAT file	N° graphe	Taille	Deg. Moy.	
2	0.0000040➤		30.00000803➤		30.000011242➤		30.000010502815➤		3	0	10	2
3	0.0000017➤		20.00000575➤		30.000012041➤		20.000011541604➤		2	1	10	2
4	0.0000020➤		30.00000803➤		30.000011781➤		30.000015802621➤		3	2	10	2
5	0.0000020➤		20.00000977➤		20.000009774➤		20.000010045289➤		2	3	10	2
6	0.0000020➤		20.00000776➤		20.000010034➤		20.000010469675➤		2	4	10	2
7	0.0000017➤		20.00000749➤		20.000009498➤		20.000010234117➤		2	5	10	2
8	0.0000017➤		30.00000722➤		30.000009029➤		30.000013513565➤		3	6	10	2
9	0.0000020➤		40.00000575➤		30.000011772➤		30.000009766817➤		3	7	10	2
10	0.0000017➤		30.00000749➤		30.000009499➤		30.000010232686➤		3	8	10	2
11	0.0000020➤		30.00000749➤		30.000009766➤		30.000009230375➤		3	9	10	2
12	0.0000020➤		30.00000750➤		30.000012047➤		30.000009498357➤		3	10	10	2
13	0.0000020➤		20.00000749➤		20.000011270➤		20.000009496927➤		2	11	10	2
14	0.0000010➤		20.00000749➤		20.000009766➤		20.000009230852➤		2	12	10	2
15	0.0000017➤		30.00000702➤		30.000011972➤		30.000009497404➤		3	13	10	2
16	0.0000034➤		20.00000646➤		20.000010068➤		20.000011537551➤		2	14	10	2
17	0.0000020➤		20.00000575➤		20.000010032➤		20.000009498357➤		2	15	10	2
18	0.0000017➤		30.00000822➤		30.000011504➤		30.000011506557➤		3	16	10	2
19	0.0000037➤		30.00000575➤		30.000010033➤		30.000011239528➤		3	17	10	2
20	0.0000017➤		30.00000849➤		30.000011236➤		30.000010968685➤		3	18	10	2

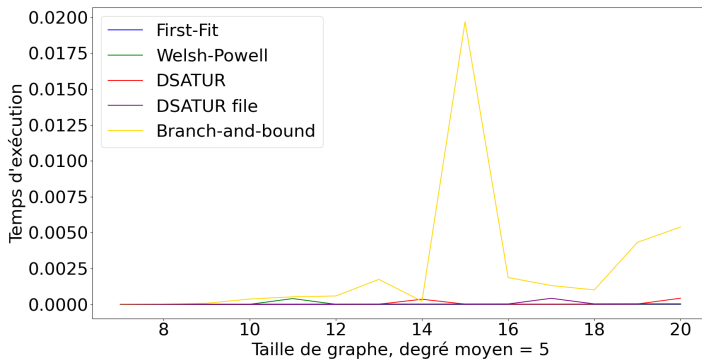
Coloration en fonction de la taille



Coloration en fonction du degré moyen

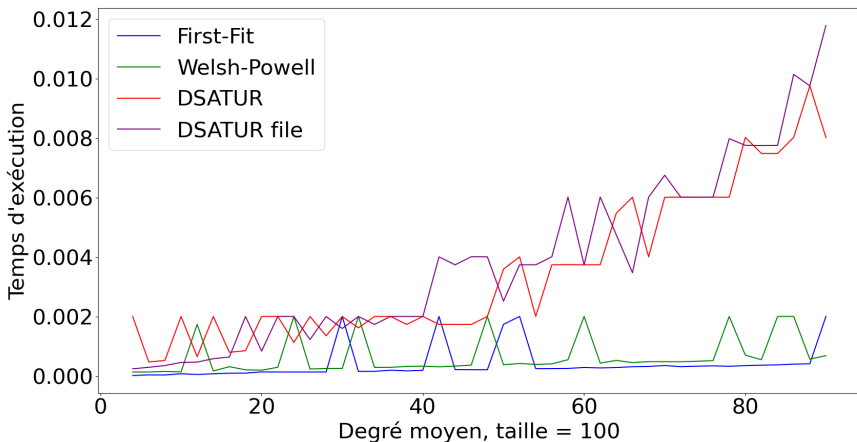


Temps d'exécution en fonction de la taille



408	15	4	20.000057433605194	3
409	15	4	30.000019901990891	3
410	15	4	40.000503865957260	4
411	15	4	50.000019533395767	3
412	15	5	10.000442697525024	4
413	15	5	20.000216532707214	4
414	15	5	30.002006769180298	4
415	15	5	40.095055818557739	5

Temps d'exécution en fonction du degré moyen



Conclusion

- Taille ≤ 20 :
 - résolution exacte possible donc à privilégier

Conclusion

- Taille ≤ 20 :
 - résolution exacte possible donc à privilégier
- Gros graphes :
 - DSATUR donne statistiquement de **bien meilleures colorations** que les autres algorithmes heuristiques
 - temps de calcul de DSATUR moins bon que Welsh-Powell et First-Fit, mais reste à privilégier car en $\mathcal{O}(n^2)$

Annexe First Fit 1

```
1 type graphe = int list array
2
3 type coloration = int array
```

Annexe First Fit 1

```

1  let first_fit (g : graphe) : coloration =
2    let n = Array.length g in
3    let colo = Array.make n (-1) in
4    let dispo = Array.make n false in
5    for s = 0 to (n-1) do      (* parcourt et colore les sommets *)
6      maj_vois g.(s)  colo  dispo ;
7      let coul = coul_min dispo in
8      colo.(s) <- coul ;
9      nettoie_vois g.(s)  colo  dispo ;
10     done;
11     colo ;;

```

```

1  (* s'exécute en  $O(n^2)$  *)

```

```

1  let rec maj_vois l  colo  dispo=
2    match l with (* marque les couleurs des voisins *)
3      | [] -> ()
4      | vois::q -> let coul_vois = colo.(vois) in
5                    if coul_vois <> -1 then
6                      dispo.(coul_vois) <- true ;
7                      maj_vois q  colo  dispo ;;

```


Annexe First Fit 2

```
1 let coul_min dispo = (* renvoie la couleur minimale disponible *)
2 let coul = ref 0 in
3 while dispo.(!coul) do
4   incr coul
5   done ;
6 !coul ;;
```

```
1 let rec nettoie_vois l colo dispo =
2 match l with (* affecte false dans dispo pour le prochain sommet *)
3   | [] -> ()
4   | vois::q -> let coul_vois = colo.(vois) in
5                 if coul_vois <> -1 then
6                   dispo.(coul_vois) <- false ;
7                 nettoie_vois q colo dispo ;;
```

Annexe Welsh-Powell

```

1  let welsh_powell (g : graphe) : coloration =
2    let n = Array.length g in
3    let colo = Array.make n (-1) in
4    let dispo = Array.make n false in
5    let fp = file_vide n in
6    for s = 0 to (n-1) do                                (*ajoute les sommets dans fp*)
7      insere fp (List.length g.(s), -s)
8    done ;
9    while not (est_vide fp) do
10     let _,som_neg = extrait_max fp in
11     let s = -som_neg in                                  (*parcourt les sommets et defile fp*)
12     let couleur = coul_min dispo in
13     (colo.(s) <- couleur ;
14      maj_vois g.(s) colo dispo ;
15      nettoie_vois g.(s) colo dispo ;
16     done ;
17     colo ;;

1  (* s'exécute en  $O(n \log(n) + m)$  *)

```

Annexe file WP 1

```
1  (*Implementation d'un tas*)
2  let file_vide n =
3    Array.make (n+1) (0,0) ;; (* fst fp.(0) = taille de la file*)

1  let est_vide fp =
2    (fst fp.(0)) = 0 ;;

1  let donnee_max fp =
2    if est_vide fp then failwith "file_vide"
3    else fp.(1) ;;

1  let echange fp i j =
2    let vi = fp.(i) in
3    fp.(i) <- fp.(j) ;
4    fp.(j) <- vi ;;
```

Annexe file WP 2

```
1  let remonter fp i =
2  let k = ref i in
3  while !k <> 1 && fp.(!k) > fp.(!k /2) do
4    échange fp !k (!k/2) ;
5    k := !k /2 ;
6  done ;;
```

```
1  let insere fp (prio,som) =
2  let taille,_ = fp.(0) in
3  if taille+1 >= Array.length fp then
4    failwith "file_pleine"
5  else
6    (fp.(0) <- (taille+1,0);
7    fp.(taille+1) <- (prio,som) ;
8    remonter fp (taille+1) ) ;;
```

Annexe file WP 3

```
1 let rec descente fp i =
2   let n = ref i in
3   let taille,_ = fp.(0) in
4   if 2*i <= taille && fp.(2*i) > fp.(i) then n := 2*i ;
5   if 2*i+1 <= taille && fp.(2*i+1) > fp.(!n) then n := 2*i+1 ;
6   if !n <> i then
7     (echange fp i !n ;
8      descente fp !n) ;;
```

```
1 let extrait_max fp =
2   if not (est_vide fp) then
3     (let taille,_ = fp.(0) in
4      fp.(0) <- (taille-1,0) ;
5      let max = fp.(1) in
6      échange fp 1 taille ;
7      descente fp 1 ;
8      max)
9   else failwith "file_vide";;
```

Annexe DSATUR 1

```

1  let dsatur (g: graphe) : coloration=
2    let n = Array.length g in
3    let colo = Array.make n (-1) in
4    let dsat_deg = Array.make n (0,0) in
5    let interdit = Array.make n [|]| in
6    for s = 0 to (n-1) do
7      let d = List.length g.(s) in
8      dsat_deg.(s) <- (0, d) ;
9      interdit.(s) <- Array.make (d + 1) false
10     let s = max_dsat dsat_deg colo in (*le sommet prioritaire *)
11     let coul = ref 0 in
12     while interdit.(s).(!coul) do
13       incr coul
14     done ;
15     colo.(s) <- !coul ;
16     maj_vois g.(s) coul interdit ;
17     done ;
18     colo ;;

1  (* s'exécute en O( n^2 ) *)

```

Annexe DSATUR 2

```
1  let max_dsat dsat_deg colo =
2  let imax = ref (-1) in
3  for sommet = 0 to (n-1) do
4    let dsat_som, deg_som = dsat_deg.(sommet) in
5    if colo.(sommet) = -1 then
6      match !imax with
7      | -1 -> imax := sommet
8      | j ->
9        let (dsat_imax,deg_imax) = dsat_deg.(j) in
10       if dsat_som > dsat_imax then
11         imax := sommet
12       else if dsat_som = dsat_imax && deg_som > deg_imax then
13         indice_max := sommet
14       else if dsat_som = dsat_imax && deg_som = deg_imax && sommet < j then
15         imax := sommet ;
16       done ;
17  !imax ;;
```

Annexe DSATUR 3

```
1  let rec maj_vois l_v coul interdit =
2    match l_v with
3    | [] -> ()
4    | vois::q ->
5      if !coul < Array.length interdit.(vois) && colo.(vois) = -1 && not interdit
        .(vois).(!coul) then then
6          (interdit.(vois).(!coul) <- true;
7          let dsat_v, deg_v = dsat_deg.(vois) in
8          dsat_deg.(vois) <- (dsat_v+1,deg_v);
9          maj_vois q interdit coul)
10     else
11       maj_vois q interdit coul ;;
```


Annexe DSATUR file 1

```

1  let dsatur_file (g : graphe) : coloration =
2    let n = Array.length g in
3    let colo = Array.make n (-1) in
4    let interdit = Array.make n [|] in
5    let hash = Array.make n (-1) in      (*table de hashage*)
6    let fp = file_vide_d n in
7    for s = 0 to (n-1) do
8      let d = List.length g.(s) in
9      interdit.(s) <- Array.make (d + 1) false ;
10     insere_d fp hash (0, d, -i)
11     done ;
12   while not (est_vide_d fp) do
13     let _, _, som_neg = extrait_max_d fp hash in
14     let s = -som_neg in
15     let coul = coul_min s interdit in
16     colo.(s) <- coul ;
17     maj_vois g.(s) coul interdit hash ;
18     done ;
19   colo ;;

1  (* s'execute en O( (n + m)log(n) ) *)

```

Annexe DSATUR file 2

```
1  let coul_min s   interdit=   (* renvoie la couleur minimale disponible *)
2      let coul = ref 0 in
3      while interdit.(s).(!coul) do
4          incr coul
5          done ;
6      !coul ;;
```

Annexe DSATUR file 3

```
1 let rec maj_vois2 l coul interdit hash =
2   match l with
3   | [] -> ()
4   | vois::q ->
5       if (colo.(vois) = -1) && not (contient interdit.(vois).(coul)) then
6         (interdit.(vois).(coul) <- true ;
7          let (dsat_v, deg_v, som_neg_v) = fp.(hash.(vois)) in
8          fp.(hash.(vois)) <- (dsat_v + 1, deg_v, som_neg_v) ;
9          remonter fp hash hash.(vois) ;
10          maj_vois2 q coul interdit hash)
11     else maj_vois q coul interdit hash ;;
```

Annexe file DSATUR 1

```
1 let file_vide_d n =
2   Array.make (n+1) (0,0,0) ;; (* fst(fp.(0)) = taille de la file*)

1 let est_vide_d fp =
2   (fst fp.(0)) = 0;;

1 let donnee_max_d fp =
2   if est_vide_d fp then failwith "file_vide"
3   else fp.(1) ;;

1 let echange_d fp hash i j =
2   let dsat_i ,deg_i ,som_neg_i = fp.(i) in
3   let _, _, som_neg_j = fp.(j) in
4   fp.(i) <- fp.(j) ;
5   fp.(j) <- dsat_i, deg_i, som_neg_i ;
6   hash.(-som_neg_i) <- j ;
7   hash.(-som_neg_j) <- i ;;
```

Annexe file DSATUR 2

```

1  let remonter_d fp hash i =
2    let k = ref i in
3    while (!k <> 1) && (fp.(!k) > fp.(!k / 2)) do
4      echange_d fp hash !k (!k/2) ;
5      k := !k / 2 ;
6    done ;;

```

```

1  let insere_d fp hash (dsat,deg,som) =
2    let taille, _, _ = fp.(0) in
3    if taille+1 >= Array.length fp then
4      failwith "file_pleine"
5    else
6      (fp.(0) <- (taille+1,0,0);
7       fp.(taille+1) <- (dsat,deg,som) ;
8       hash.(-som) <- taille+1 ;
9       remonter_d fp hash (taille+1) ) ;;

```

Annexe file DSATUR 3

```

1  let rec descente_d fp hash i =
2    let n = ref i in
3    let taille,_,_ = fp.(0) in
4    if 2*i <= taille && fp.(2*i) > fp.(i) then
5      n := 2*i ;
6    if 2*i+1 <= taille && fp.(2*i+1) > fp.(!n) then
7      n := 2*i+1 ;
8    if !n <> i then
9      (echange_d fp hash i !n;
10       descente_d fp hash !n)

```

```

1  let extrait_max_d fp hash =
2    if not (est_vide_d fp) then
3      (let taille,_,_ = fp.(0) in
4       fp.(0) <- (taille-1,0,0) ;
5       let max = fp.(1) in
6       (echange_d fp hash 1 taille ;
7        descente_d fp hash 1 ;
8        max)
9    else failwith "file_vide" ;;

```

Annexe B&B 1

```

1  let branch_and_bound  g =
2    let n = Array.length graphe in
3    if n = 0
4      then [||],0
5    let xmax, colo_min = init_colo  n in
6    let colo= Array.make n (-1) in (* la colo modifiable *)
7    colo.(0) <- 0 ;                (*initialisation *)
8    let s = ref 0 in                (*dernier indice colore valide *)
9    let rec bab  colo  =
10      if colo.(0) = -1 then !colo_opti,(!xmax +1) (* fin du parcourt*)
11    else
12      if !s = n-1 then              (* colo finie -> ameliorree *)
13        maj_colo_min  colo  colo_opti  xmax  n
14      else
15        colo.(!s +1) <- 0 ;
16        if not (continue  colo  (!s + 1)) then
17          s := next_colo  colo  (!s + 1)  xmax
18        else
19          s := !s + 1 ;
20          bab  colo
21  in bab  colo;;

```

Annexe B&B 2

```

1 let init_colo n =
2   let colo_i = Array.make n 0 in
3   for i = 1 to (n-1) do
4     colo_i.(i) <- i
5   done ;
6   (ref colo_min), (ref (n-1)) ;;

```

```

1 let continue colo s xmax g = (*fonction de rejet *)
2   let rec est_valide l =
3     match l with
4     | [] -> true
5     | vois :: q -> if colo.(vois) = colo.(s) && vois < s
6                     then false
7                     else est_valide q
8   in (colo.(s) < !xmax) && (est_valide g.(s)) ;;

```


Annexe B&B 3

```
1 let rec next_colo colo i0 xmax =  
2   let i = ref i0 in  
3   while !i >= 0 && colo.(!i) = xmax do  
4     colo.(!i) <- -1 ;  
5     decr i ;  
6     done ;  
7     if !i >= 0 then  
8       colo.(!i) <- colo.(!i) +1 ;  
9       if continue colo !i  
10        then !i  
11        else next_colo colo !i )  
12   else !i ;;
```

Annexe B&B 4

```
1 let maj_colo_min colo colo_opti xmax n =
2   let coul_max = ref 0 in
3   let imax = ref 0 in
4   for i = 0 to (n-1) do
5     if colo.(i) > !coul_max then
6       coul_max := colo.(i) ;
7       imax := i ;
8   done ;
9   colo_opti := Array.copy colo ;
10  xmax := !coul_max ;
11  for j= !imax to (n-1) do
12    colo.(j) <- -1 ;
13  done ;
14  s := next_colo colo !imax ;;
```

Annexe graphe aléatoire 1

```
1 Random.self_init () ;;
2 let max = 1073741823 ;; (* c'est 2^30 -1, le + grand int *)
3
4 let construit_g_rand t proba = (* en  $O(n^2)$  *)
5   let graphe = Array.make t [] in
6   for i = 0 to (t-1) do
7     for j = (i+1) to (t-1) do
8       let n = Random.int max in
9       if n < int_of_float ( (float_of_int max) *. proba) then
10         (graphe.(i) <- j::graphe.(i) ;
11          graphe.(j) <- i::graphe.(j))
12     done ;
13   done ;
14   graphe ;;
```

Annexe graphe aléatoire 2

```
1 Random.self_init () ;;
2
3 let construit_g_rand_eco t nb_arete =
4   let graphe = Array.make t [] in
5   let table = Hashtbl.create (t*t) in
6   let rec rempli nb_arete =
7     if nb_arete > 0 then
8       let i = Random.int t in
9       let j = Random.int t in
10      if i <> j && not (Hashtbl.mem table (i,j)) then
11        (Hashtbl.add table (i,j) 1 ;
12         Hashtbl.add table (j,i) 1 ;
13         graphe.(i) <- j::graphe.(i) ;
14         graphe.(j) <- i::graphe.(j)) ;
15      rempli (nb_arete -1)
16   in rempli nb_arete ;
17 graphe ;;
```

Annexe répertoire

```
1  let liste_couples =
2    [ 10,2 ; 10,4 ; 10,6 ; 10,8 ;
3      12,2 ; 12,4 ; 12,6 ; 12,8 ;
4      15,2 ; 15,4 ; 15,6 ; 15,8 ; 15,10 ; 15,12 ] ;;
5    (* couples taille-degre moyen *)
6
7  let g c = match c with
8    |(10,2) -> g_10_2
9    |(10,4) -> g_10_4
10   |(10,6) -> g_10_6
11   |(10,8) -> g_10_8
12   |(12,2) -> g_12_2
13   |(12,4) -> g_12_4
14   |(12,6) -> g_12_6
15   |(12,8) -> g_12_8
16   |(15,2) -> g_15_2
17   |(15,4) -> g_15_4
18   |(15,6) -> g_15_6
19   |(15,8) -> g_15_8
20   |(15,10) -> g_15_10
21   |(15,12) -> g_15_12 (* ici ne figure qu'un extrait *)
```

Annexe fichier de graphes 1

```

1  let écrire_texte3 t nom destination =
2    let n = Array.length t in
3    output_string destination ("let_") ;
4    output_string destination (nom) ;
5    output_string destination ("_=[") ;
6    for i = 0 to (n-2) do
7      écrit_tableau t.(i) ;
8      output_string destination (";")
9    done ;
10   if n >= 1 then
11     écrit_tableau t.(n-1) ;
12   output_string destination ("|]") ;;

```

```

1  let rec aux l destination = match l with
2  | [] -> output_string destination ("]")
3  | [e] -> (output_string destination (string_of_int e) ;
4            output_string destination ("]") )
5  | e::q -> (output_string destination (string_of_int e) ;
6            output_string destination (";") ; aux q destination) ;;

```

Annexe fichier de graphes 2

```
1 let ecrit_liste l destination =  
2   output_string destination "[" ;  
3   aux l ;;
```

```
1 let ecrit_tableau t destination =  
2   let n = Array.length t in  
3   output_string destination "[" ;  
4   for i = 0 to (n-2) do  
5     ecrit_liste t.(i) destination ;  
6     output_string destination ";" ;  
7   done ;  
8   if n >= 1 then  
9     ecrit_liste t.(n-1) ;  
10  output_string destination "]" ;;
```

Annexe fichier de graphes 3

```
1 let ecrire_tout l_t_g l_nom fichier =
2   let destination = open_out fichier in
3   let rec aux lg lnom = match lg,lnom with
4     | [],[] -> ()
5     | t_g::qg, nom::qnom ->
6       (ecrire_texte3 t_g nom destination ;
7        output_string destination ("␣;;" ^ "\n" ^ "\n") ;
8        aux qg qnom)
9     | _,_ -> ()
10  in
11  aux l_t_g l_nom ;
12  close_out destination ;;
```


Annexe fichier de graphes 4

```

1  (*convertit une liste de couples representant une categorie en un string qui
   dit la meme chose*)
2  let rec liste_couple_to_nom lc = match lc with
3  | [] -> []
4  |(taille,densite)::q -> ("g_" ^ (string_of_int taille) ^ "_" ^ (string_of_int
   densite)) :: (liste_couple_to_nom q)

```

```

1  (*constitue un tableau de graphes pour une categorie fixee*)
2  let construit_categorie t d n =
3  let a = Array.make n [[]] in
4  for i = 0 to (n-1) do
5    a.(i) <- construit_graphe_random_eco t (t*d/2)
6  done ;
7  a ;;

```

```

1  (*genere une categorie de graphes etant donnees les parametres *)
2  let rec liste_couple_to_graphe lc n = match lc with
3  | []->[]
4  |(taille,densite)::q ->
5  (construit_categorie taille densite n)::(liste_couple_to_graphe q n)

```

Annexe fichier de graphes 5

```
1  (*fonction finale, rassemble tout, c'est ca qu'on lance a la fin, ou lc est la
    liste des categories de graphes a construire*)
2
3  let fonction_finale lc n fichier =
4    ecrire_tout (liste_couple_to_graphe lc n) (liste_couple_to_nom lc) fichier
```

Conversion csv 1

```
1 let file = "donnees.csv";;
2
3 let tableau_tps l list_test list_fonction =
4   let oc = open_out file in
5   let lg = creation_test l in
6   let rec aux1 lg l = match lg,l with
7     | [],_ -> ()
8     | g::qg,(t,a)::qc ->
9       let nb_graphe = Array.length g in
10      for i = 0 to (nb_graphe - 1) do
11        imprime list_fonction oc i t a;;
12      done ;
13   in aux1 lg l ;
14   close_out_noerr oc ;;
```

Conversion csv 1

```
1 let rec imprime lf oc i t a =
2   match lf with
3   | [] ->
4     Printf.fprintf oc "%i," i;
5     Printf.fprintf oc "%i" t;
6     Printf.fprintf oc "%s" ", ";
7     Printf.fprintf oc "%i\n" a;
8   | f::qf ->
9     let debut = Unix.gettimeofday () in
10    let coloration = f g.(i) in
11    let fin= Unix.gettimeofday () in
12    Printf.fprintf oc "%.15f," (fin-.debut) ;
13    Printf.fprintf oc "%i," (max coloration) ;
14    imprime qf oc i t a ;;
```

Annexe courbes 1

```
1 import csv
2 import matplotlib.pyplot as plt
3 import numpy as np
4 file = "D:/MPET/TIPE/Fichiers_de_code/final.csv"
5 tps_naif = [] ; colo_naif = []
6 tps_welsh_powel = [] ; colo_welsh_powel = []
7 tps_dsatur = [] ; colo_dsatur = []
8 tps_dsatur_file = [] ; colo_dsatur_file = []
9 tps_bb = [] ; colo_bb = []
10 taille_g = []
11 aretes_g = []
12 l_cat = [ (10,2) , ( 10,4 ) , ( 10,6 ) , ( 10,8 ) , (
13     12,2 ) , ( 12,4 ) , ( 12,6 ) , ( 12,8 ) , (
14     15,2 ) , ( 15,4 ) , ( 15,6 ) , ( 15,8 ) ] #liste non exhaustive
```

Annexe courbes 1

```
1 with open(file, 'r', encoding='utf-8') as csvfile:
2     reader = csv.reader(csvfile, delimiter = ',')
3     for row in reader:
4         if row[0] != 't':
5             tps_naif.append(float(row[0]))
6             colo_naif.append(int(row[1]))
7             tps_welsh_powel.append(float(row[2]))
8             colo_welsh_powel.append(int(row[3]))
9             tps_dsatur.append(float(row[4]))
10            colo_dsatur.append(int(row[5]))
11            tps_dsatur_file.append(float(row[6]))
12            colo_dsatur_file.append(int(row[7]))
13            tps_backtracking.append(float(row[8]))
14            colo_backtracking.append(int(row[9]))
15            taille_g.append(int(row[9]))
16            aretes_g.append(int(row[10]))
```

Annexe courbes 2

```
1 def liste_categorie(s,a):
2     c = rech_ind((s,a))
3     l = [c*100 + i for i in range(100)]
4     return l
```

```
1 def moyenne(couple, atraiter):
2     l = liste_categorie(s,a)
3     s = 0
4     for i in l:
5         s = s + atraiter[i]
6     return (s/(len(l)))
```

Annexe courbes 3

```
1 dico_d = {}
2 for i in range(len(l_cat)):
3     t,d = l_cat[i]
4     if t <= 400 : #fin du tableau excell a prendre en compte, a changer a la
        main
5         if not d in dico_d :
6             dico_d[d] = [l_cat[i]]
7         else :
8             dico_d[d].append(l_cat[i])

1 dico_t = {}
2 for i in range(len(l_cat)):
3     t,d = l_cat[i]
4     if t <= 400 : #fin du tableau excell a prendre en compte, a changer a la
        main
5         if not t in dico_t :
6             dico_t[t] = [l_cat[i]]
7         else :
8             dico_t[t].append(l_cat[i])
```


Annexe courbes 4

```
1 def nom(fonc):
2     if fonc == 'n' : return 'First-Fit'
3     if fonc == 'wp' : return 'Welsh-Powell'
4     if fonc == 'd' : return 'DSATUR'
5     if fonc == 'df' : return 'DSATUR_file'
6     if fonc == 'b' : return 'Branch-And-bound'

1 def t(fonc):
2     if fonc == 'n' : return tps_naif,colo_naif
3     if fonc == 'wp' : return tps_welsh_powel,colo_welsh_powel
4     if fonc == 'd' : return tps_dsatur,colo_dsatur
5     if fonc == 'df' : return tps_dsatur_file,colo_dsatur_file
6     if fonc == 'b' : return tps_bb,colo_bb
```

Annexe courbes 5

```
1 def moy_nc(c,fonc):
2     tps,colo = t(fonc)
3     moy = 0
4     d,f = deb_fin(c)
5     for i in range(d,f):
6         moy = moy + colo[i]
7     moy = moy/100
8     return moy
```

```
1 def moy_nc_t(c,fonc):
2     tps,colo = t(fonc)
3     moy = 0
4     d,f = deb_fin(c)
5     for i in range(d,f):
6         moy = moy + tps[i]
7     moy = moy/100
8     return moy
```

Courbes temps-degré moyen 1

```
1 def comp_tps_dm(dm,f1,f2,f3,f4):
2     plt.close()
3     t1 = [] ; t2 = [] ; t3 = [] ; t4 = []
4     lc = dico_d[dm]
5     tm = []
6     tailles = []
7     for c in lc :
8         t,d = c
9         m1 = moy_nc_t(c,f1)
10        m2 = moy_nc_t(c,f2)
11        m3 = moy_nc_t(c,f3)
12        m4 = moy_nc_t(c,f4)
13        t1.append(m1)
14        t2.append(m2)
15        t3.append(m3)
16        t4.append(m4)
17        tm.append(m1+m2+m3+m4)
18        tailles.append(t)
```

Courbes temps-degré moyen 2

```
1 plt.figure()
2 plt.plot(tailles,t1,ls = '-', marker='', color = 'blue', label = nom(f1))
3 plt.plot(tailles,t2,ls = '-', marker='', color = 'green', label = nom(f2))
4 plt.plot(tailles,t3,ls = '-', marker='', color = 'red', label = nom(f3))
5 plt.plot(tailles,t4,ls = '-', marker='', color = 'purple', label = nom(f4))
6
7 plt.xlabel('Taille, _degre_moyen_='+ str(dm), fontsize = 20)
8 plt.ylabel("Temps_d'execution", fontsize = 20)
9 plt.xticks(fontsize = 20)
10 plt.yticks(fontsize = 20)
11 plt.legend(loc=2, prop={'size': 20})
plt.show()
```

Courbes temps-taille 1

```
1 def comp_tps_t(ta,f1,f2,f3,f4):
2     plt.close()
3     t1 = [] ; t2 = [] ; t3 = [] ; t4 = []
4     lc = dico_t[ta]
5     tailles = []
6     for c in lc :
7         t,d = c
8         m1 = moy_nc_t(c,f1)
9         m2 = moy_nc_t(c,f2)
10        m3 = moy_nc_t(c,f3)
11        m4 = moy_nc_t(c,f4)
12        t1.append(m1)
13        t2.append(m2)
14        t3.append(m3)
15        t4.append(m4)
16        tailles.append(d)
```

Courbes temps-taille 2

```
1 plt.figure()
2 plt.plot(tailles,t1,ls = '-', marker='', color = 'blue', label = nom(f1))
3 plt.plot(tailles,t2,ls = '-', marker='', color = 'green', label = nom(f2))
4 plt.plot(tailles,t3,ls = '-', marker='', color = 'red', label = nom(f3))
5 plt.plot(tailles,t4,ls = '-', marker='', color = 'purple', label = nom(f4))
6
7 plt.xlabel('Degre_moyen, taille'+ str(ta), fontsize = 20)
8 plt.ylabel("Temps_d'execution", fontsize = 20)
9 plt.xticks(fontsize = 20)
10 plt.yticks(fontsize = 20)
11 plt.legend(loc=2, prop={'size': 20})
12 plt.show()
```

Courbes coloration-degré moyen 1

```
1 def comp_nc_dm(dm,f1,f2,f3,f4):
2     plt.close()
3     nc1 = [] ; nc2 = [] ; nc3 = [] ; nc4 = []
4     nc1sur3 = [] ; nc2sur3 = [] ; nc4sur3 = []
5     lc = dico_d[dens]
6     tailles = []
7     for c in lc :
8         t,d = c
9         m1 = moy_nc(c,f1)
10        m2 = moy_nc(c,f2)
11        m3 = moy_nc(c,f3)
12        m4 = moy_nc(c,f4)
13        nc1.append(m1)
14        nc2.append(m2)
15        nc3.append(m3)
16        nc4.append(m4)
17        nc1sur3.append(m1/m3)
18        nc2sur3.append(m2/m3)
19        nc4sur3.append(m4/m3)
20        tailles.append(t)
```

Courbes coloration-degré moyen 2

```
1 plt.figure()
2 plt.plot(tailles,nc1,ls = '--', marker='', color = 'blue', label = nom(f1))
3 plt.plot(tailles,nc2,ls = '--', marker='', color = 'green', label = nom(f2)
4 )
5 plt.plot(tailles,nc3,ls = '--', marker='', color = 'red', label = nom(f3))
6 plt.plot(tailles,nc4,ls = '--', marker='', color = 'purple', label = nom(f4
7 ))
8 plt.xlabel('Taille, édegré_moyen_'+ str(dm), fontsize = 20)
9 plt.ylabel('Coloration_moyenne', fontsize = 20)
10 plt.xticks(fontsize = 20)
11 plt.yticks(fontsize = 20)
12 plt.legend(loc=2, prop={'size': 20})
13 plt.show()
```


Courbes coloration-degré moyen 3

```
1 plt.figure()
2 plt.plot(tailles,[1]*len(tailles), color = 'lightgrey')
3 plt.plot(tailles,nc1sur3,ls = '--', marker='', color = 'blue', label = nom(
    f1) + ' / ' + nom(f3))
4 plt.plot(tailles,nc2sur3,ls = '--', marker='', color = 'green', label = nom
    (f2) + ' / ' + nom(f3))
5 plt.plot(tailles,nc4sur3,ls = '--', marker='', color = 'purple', label =
    nom(f4) + ' / ' + nom(f3))
6 plt.ylim([0.95, 1.45])
7 plt.xlabel('Taille de graphe, degré moyen = ' + str(dm), fontsize = 20 )
8 plt.ylabel('Nb coul / Nb coul ' + nom(f3), fontsize = 20)
9 plt.xticks(fontsize = 20)
10 plt.yticks(fontsize = 20)
11 plt.legend(loc=2, prop={'size': 20})
12 plt.show()
```

Courbes coloration-taille 1

```
1 def comp_nc_t(ta,f1,f2,f3,f4):
2     plt.close()
3     nc1 = [] ; nc2 = [] ; nc3 = [] ; nc4 = []
4     nc1sur3 = [] ; nc2sur3 = [] ; nc4sur3 = []
5     lc = dico_t[ta]
6     tailles = []
7     for c in lc :
8         t,d = c
9         m1 = moy_nc(c,f1)
10        m2 = moy_nc(c,f2)
11        m3 = moy_nc(c,f3)
12        m4 = moy_nc(c,f4)
13        nc1.append(m1)
14        nc2.append(m2)
15        nc3.append(m3)
16        nc4.append(m4)
17        nc1sur3.append(m1/m3)
18        nc2sur3.append(m2/m3)
19        nc4sur3.append(m4/m3)
20        tailles.append(d)
```

Courbes coloration-taille 2

```
1 plt.figure()
2 plt.plot(tailles,nc1,ls = '--', marker='', color = 'blue', label = nom(f1))
3 plt.plot(tailles,nc2,ls = '--', marker='', color = 'green', label = nom(f2)
4 )
5 plt.plot(tailles,nc3,ls = '--', marker='', color = 'red', label = nom(f3))
6 plt.plot(tailles,nc4,ls = '--', marker='', color = 'purple', label = nom(f4
7 ))
8 plt.xlabel('éDensité de graphe, taille=' + str(ta), fontsize = 20 )
9 plt.ylabel('Coloration moyenne', fontsize = 20 )
10 plt.xticks(fontsize = 20)
11 plt.yticks(fontsize = 20)
12 plt.legend(loc=2, prop={'size': 20})
13 plt.show()
```

Courbes coloration-taille 3

```

1  plt.figure()
2  plt.plot(tailles,[1]*len(tailles), color = 'lightgrey')
3  plt.plot(tailles,nc1sur3,ls = '--', marker='', color = 'blue', label = nom(
      f1) + ' / ' + nom(f3))
4  plt.plot(tailles,nc2sur3,ls = '--', marker='', color = 'green', label = nom
      (f2) + ' / ' + nom(f3))
5  plt.plot(tailles,nc4sur3,ls = '--', marker='', color = 'purple', label =
      nom(f4) + ' / ' + nom(f3))
6  plt.ylim([0.95, 1.45])
7  plt.xlabel('éDensité du graphe, taille = ' + str(ta), fontsize = 20 )
8  plt.ylabel('Nb coul / Nb coul' + nom(f3) , fontsize = 20)
9  plt.xticks(fontsize = 20)
10 plt.yticks(fontsize = 20)
11 plt.legend(loc=2, prop={'size': 20})
12 plt.show()

```

SAT

Soit $x_{i,c}$ la variable : le sommet i a la couleur c

Φ_1 : Chaque sommet a au moins une couleur

$$\Phi_1 = \bigwedge_{i \in V} \left(\bigvee_{c=1}^k x_{i,c} \right)$$

Φ_2 : Chaque sommet a au plus une couleur

$$\Phi_2 = \bigwedge_{i \in V} \left(\bigwedge_{1 \leq c < c' \leq k} (\neg x_{i,c} \vee \neg x_{i,c'}) \right)$$

SAT

Φ_3 : Contrainte de conflit (Arêtes)

$$\Phi_3 = \bigwedge_{(i,j) \in E} \left(\bigwedge_{c=1}^k (\neg x_{i,c} \vee \neg x_{j,c}) \right)$$

Formule de Colorabilité Φ

$$\Phi = \Phi_1 \wedge \Phi_2 \wedge \Phi_3$$